Ders 5 Genişletilmiş Notu: Model Değerlendirme, Çapraz Doğrulama ve Başarı Ölçüleri

Ders 5 Genişletilmiş Notu: Model Değerlendirme, Çapraz Doğrulama ve Başarı Ölçüleri

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Neden Modelin Başarısını Ölçmeliyiz?
- 2. Genelleme: İnsan Gibi Öğrenmek
- 3. Üç Model, Üç Karar Sınırı
- 4. Aşırı Öğrenme ve Yetersiz Öğrenme
- 5. Model Karmaşıklığı ve Occam'ın Usturası
- 6. Birden Fazla Model, Birden Fazla Parametre
- 7. Altın Kural: Eğitim, Doğrulama ve Test Verisi
- 8. Az Veri Problemi
- 9. Holdout Yöntemi
- 10. Rastgele Alt Örnekleme
- 11. Eğitim Örneklerine Müdahale Etmemek
- 12. Tabakalı Örnekleme (Stratification)
- 13. K Katlı Çapraz Doğrulama
- 14. Birini Dışarıda Bırak ve Bootstrap
- 15. Son Model: Bütün Veriyle Yeniden Eğitmek
- 16. Dengesiz Veri
- 17. Karışıklık Matrisi
- 18. Çok Sınıflı Problemlerde Karışıklık Matrisini Okumak
- 19. Başarı Ölçüleri: Accuracy, Precision, Recall, Specificity, F1
- 20. ROC Eğrisi ve AUC

1. Neden Modelin Başarısını Ölçmeliyiz?

Hoca derse, şimdiye kadar gördüğümüz eğitmenli öğrenme (supervised learning) yöntemlerini hatırlatarak başladı: karar ağaçları, Bayes sınıflandırıcısı ve K en yakın komşu (k-NN). Karar ağaçlarını görürken bir sorundan bahsetmiştik: Ağacın seviyesi çok artarsa aşırı öğrenme (overfitting) olur, yani eğitim başarısı yüksek, test başarısı düşük çıkar. Bunu önlemek için ağacı budamak gerektiğini söylemiştik. Fakat o sırada açıkta kalan bir soru vardı: Bir model oluşturduk, peki bu modelin başarısı nedir, nasıl ölçeceğiz, nasıl iyileştireceğiz?

Bir modeli eğittikten sonra asıl önemli olan, o modelin ne kadar başarılı olduğudur. Çünkü modeli kullanıp kullanmamaya buna göre karar veririz. Başarı ölçümü iki farklı kararda gerekir:

- **Farklı modeller arasında seçim:** Karşılaştığımız bir problemde karar ağacı mı, k-NN mi, Bayes mi kullanacağız? Bunun için modellerin başarısı ölçülür ve en iyisi seçilir.
- **Aynı modelin parametrelerini seçme:** Örneğin k-NN'de K'yi 3 mü, 5 mi, 7 mi almalıyız? En iyi sınıflandırmayı veren K değerini bulmak için de modelin başarısını ölçmemiz gerekir.

Bu derste bu değerlendirmenin nasıl yapılması gerektiği anlatıldı.

2. Genelleme: İnsan Gibi Öğrenmek

Hocaya göre makine öğrenmesinde yapmaya çalıştığımız şey, insanların öğrenmesindekiyle aynıdır: genelleme (generalization) yapmak. Gerçek dünyada karşılaştığımız olaylardan genel kurallar çıkarırız. Hiç bilmediğimiz bir olayla karşılaştığımızda da bu bilgileri kullanarak tahmin yapar, karar veririz.

Hocanın örneği kuşlardı. Zaman içinde kuşların kanatlı olduğunu, gagaları ve iki ayakları olduğunu, bu ayakların özelliklerini öğreniriz ve bir genelleme yaparız. Hiç görmediğimiz tropik bir kuşla karşılaştığımızda, aklımızdaki kuş imajı sayesinde bunun da bir kuş olduğunu anlarız. Yani öğrendiğimiz örneklerden genel özellikleri çıkarırız. Bir öğrenme modelinden beklenen de budur.

3. Üç Model, Üç Karar Sınırı

Hoca slaytta iki sınıflı bir problem gösterdi: iki balık türü, somon (salmon) ve levrek (sea bass). İki özellik kullanılıyordu: balığın uzunluğu ve pullarının parlaklığı. Bu iki özelliğe göre üç farklı modelin iki sınıfı nasıl ayırdığı gösterilmişti.

- **Birinci model:** Sınıfları doğrusal (linear) bir karar sınırıyla ayırıyor. Karar sınırının iki tarafında da o tarafa ait olmayan birkaç örnek var. Hoca bunun aslında oldukça başarılı bir model olduğunu söyledi.
- **İkinci model:** Bütün eğitim örneklerini doğru sınıflandırıyor. Bütün somonlar bir tarafta, bütün levrekler öbür tarafta. İlk bakışta "şahane" bir model.
- **Üçüncü model:** Doğrusaldan daha esnek bir karar sınırı. Yine iki tarafta da birkaç yanlış taraftaki örnek var, ama sınır sınıfların genel yapısına daha uygun.

Hoca önce "en iyi" görünen ikinci modeli inceledi. Eğitim örneklerinin hepsi doğru yerde. Fakat slaytta bir soru işaretiyle gösterilen, sınıfı bilinmeyen yeni bir test örneği var. İkinci model bu örneğin levrek sınıfına ait olduğunu söylüyor. Oysa örneğin bulunduğu bölgeye bakınca, çevresindeki örneklerin çoğu somon. Burada birkaç en yakın komşuya bakan bir k-NN kullansaydık, örneği muhtemelen somon olarak sınıflandırırdık. O bölgede yalnızca, levreklerin genel özelliklerine uymayan, öbür tarafa doğru "kaçmış" birkaç örnek var.

Sorun şudur: Model eğitim örneklerini en iyi şekilde sınıflandırmaya odaklandığı için ezberlemiştir. Buna aşırı öğrenme (overfitting) denir. Eğitim örnekleri çok yüksek başarıyla sınıflandırılırken, bilinmeyen örneklerde başarı düşer. Model oluştururken kaçınmamız gereken durum budur. İyi bir model, üçüncü modelde olduğu gibi en iyi genellemeyi sağlamalıdır.

4. Aşırı Öğrenme ve Yetersiz Öğrenme

Hoca ardından üç modeli bir kez daha, bu kez öğrenme durumuna göre adlandırarak gösterdi:

- **Yetersiz öğrenme (underfitting):** Model yeterince iyi öğrenilmemiş.
- **Aşırı öğrenme (overfitting):** Model "fazlasıyla iyi" öğrenmiş. Bütün eğitim örnekleri doğru sınıflandırılana kadar öğrenme devam etmiş. Buna ezberleme deriz.
- **Uygun model:** Esnek ama ezberlemeyen bir karar sınırı. Farklı sınıflardan örnekler yine sınırın diğer tarafında kalabilir, ama sınır modelin genel yapısını daha iyi gösterir.

4.1 Aşırı Öğrenme Nasıl Anlaşılır, Neden Olur?

Bir model oluşturup başarısını ölçtüğünüzde **eğitim başarısı test başarısından çok yüksek çıkıyorsa** aşırı öğrenmeden şüphelenmelisiniz. Model eğitim örneklerini çok iyi öğrenecek şekilde eğitilmeye devam edilmiştir. Artık genelleme değil ezberleme vardır ve test örneklerinde hata yüksek çıkar.

Hoca aşırı öğrenmenin farklı sebeplerini saydı:

- **Geneli temsil etmeyen örnekler:** Az önceki balık örneğinde olduğu gibi, verinin içinde sınıfının genel özelliklerine uymayan örnekler olabilir. Bunlar gürültüden (parazit) kaynaklanabilir. Ama gürültü olmadan da olabilir. Hocanın örneği: Bir sınıftaki kız öğrencilerin içinde 1,90 boyunda, 100 kilo bir öğrenci de olabilir. Bu bir parazit değildir, anomali de demeyelim; sadece geneli temsil etmeyen bir durumdur. Böyle bir örneğin varlığı tek

başına aşırı öğrenmeye sebep olmaz. Sebep, o örnek de doğru sınıflandırılana kadar eğitime devam etmektir.
- **Gereğinden fazla özellik:** Ayırt edici özelliklerle yetinmeyip, ayırt ediciliği olmayan, gereksiz özelliklerle de öğrenmeye devam etmek aşırı öğrenmeye yol açabilir.

4.2 Yetersiz Öğrenme

Yetersiz öğrenme, öğrenme tamamlanmadan öğrenmeyi kesmektir. Hoca bunun "öğrenme tamamlanmadan nasıl kesilir ki?" diye garip gelebileceğini, ama farkında olmadan sık yapıldığını söyledi.

Bazı öğrenme modellerinde öğrenme yinelemeyle (iteration) yapılır ve bir durdurma ölçütü seçilir. Örneğin bir yapay sinir ağında iki seçenek vardır:

- Hata belli bir eşik değerinin, yani bir ϵ değerinin altına düşene kadar öğren.
- Belli bir adım sayısı kadar öğren, örneğin 20 bin adım.

Adım sayısını sabit verirseniz ve o problem için 20 bin adım yetmiyorsa, yetersiz öğrenme ile karşılaşırsınız.

4.3 Bir Anı: Sesle Fare Kontrolü

Hoca bunu akılda kalıcı kılmak için asistanlığı sırasında yaşadıkları bir olayı anlattı. Yıllar önce iki lisans öğrencisinin bitirme projesini yönetiyorlardı. O zaman bugünkü gibi hazır ses tanıma araçları (tool) yoktu; tanıma işini öğrencilerin kendileri yazması gerekiyordu. Proje, farenin sesle kontrolüydü: "yukarı" deyince fare imleci yukarı, "aşağı" deyince aşağı gidiyor; sağ, sol ve "tıkla" komutları da vardı.

Öğrenciler sınıflandırma için klasik bir yapay sinir ağı kullanıyordu. Özellik olarak ses için Mel frekans kepstral katsayıları (Mel Frequency Cepstral Coefficients, MFCC) kullanılıyordu. Fakat başarı bir türlü yükselmiyordu. Hocanın hatırladığı kadarıyla literatürde %80 civarında başarı beklenirken, onlarınki %60'ı geçmiyordu.

Uzun süre başka yerlerde sorun aradılar: Eğitim örnekleri mi iyi değil, yapılan ses kayıtları mı kötü, kodda mı hata var? Bir sürü olasılığın üzerinden geçtiler. Model basit olduğu ve kodu da incelediği için modelin doğruluğundan emindiler. Sonunda kodu iyice dikkatle incelerken şunu fark ettiler: Öğrenciler durdurma ölçütü olarak "hata ϵ değerinin altına düşünce dur" yerine sabit bir adım sayısı kullanmışlardı. Verdikleri adım sayısı o model için öğrenmeye yetmemişti. Yani yetersiz öğrenme vardı.

Hoca bunun gerçekten farkına varılmadan karşılaşılan bir durum olduğunu söyledi. Çünkü yineleme gerektiren çalışmalarda "düşük hataya ulaşmak zor olacak, ϵ'u beklemeyeyim, adım sayısı vereyim" denerek bu yol sık kullanılır.

4.4 Çözümler

Yetersiz öğrenmenin genel sebebi, öğrenmenin tamamlanmaması, genellemeye ulaşılmadan kesilmesidir. Hocaya göre çözümü kolaydır: Modeli daha çok eğitirsiniz. Ya da gerçekten o model o problemi öğrenemiyordur; bunu da bu şekilde anlayıp çözebilirsiniz.

Aşırı öğrenmenin tipik örneği karar ağaçlarındaydı. Hoca slaytta, düğüm sayısına göre eğitim ve test başarısını gösteren bir grafik gösterdi. Düğüm sayısı arttıkça eğitim başarısı artmaya devam ediyor. Test başarısı ise belli bir noktadan sonra düşmeye başlıyor. Çünkü düğüm sayısının artması, eğitim örneklerindeki her durumun (case) ayrı ayrı öğrenilmesi demektir. Bilinmeyen test örneklerinde ise genelleme yapılamadığı için sorun çıkar. Grafikte gürültülü bir noktanın da öğrenilmeye çalışıldığı görülüyor.

5. Model Karmaşıklığı ve Occam'ın Usturası

Hocanın bu bölümdeki ana fikri şudur: Model ne kadar karmaşık olursa, eğitimde o kadar az hata yaparsınız. Çünkü modelin karmaşık olması, her şeyi öğrenmeye çalışması demektir. Sıfır eğitim hatası bile elde edebilirsiniz, ama test hatanız yüksek olur. Sebebi, genelleme yapılamamasıdır.

Hoca bunu balık örneğine bağladı: Karar sınırı, sınıfının genelinden kopmuş tek bir noktayı da öğrenecek şekilde çizilirse, o noktanın yakınına gelen bilinmeyen bir örnek ister istemez yanlış sınıfa atanır. Aşırı öğrenmenin sıkıntısı budur.

Makine öğrenmesinde çok sık referans verilen bir ilke burada devreye girer: **Occam'ın usturası (Occam's razor).** Genelleme hatası birbirine benzeyen iki modeliniz varsa, basit olanı seçin. Yani her zaman çok karmaşık olmayan, ezberlemeye gitmeyen bir model elde etmeye çalışılmalıdır.

6. Birden Fazla Model, Birden Fazla Parametre

Peki bunu nasıl yapacağız, nasıl anlayacağız? Hoca önce mutlaka dikkat edilmesi gereken bir noktayı vurguladı.

> **Hocanın vurgusu:** Bir makine öğrenmesi sistemi kurarken mutlaka birden fazla model denenmelidir.

Hiçbir zaman şöyle yapmayın: "K en yakın komşuyu denedim, %90 başarı verdi, bunu kullanayım." Bu doğru bir yaklaşım değildir. Belki karar ağacı kullansaydınız %93 başarı elde edecektiniz. Sizin probleminiz için o %3'lük fark büyük bir fark olabilir. Bu yüzden iki şey yapılmalıdır:

1. Birden fazla modeli hazırlayıp bunların içinden en başarılısını seçin.
2. Aynı model için farklı parametreleri deneyip en iyi parametreli modeli seçin. Örneğin k-NN'de K değeri, karar ağacında ağacın yüksekliği gibi.

7. Altın Kural: Eğitim, Doğrulama ve Test Verisi

Ölçme işlemi için elimizdeki veri iki parçaya ayrılır: eğitim (training) ve test. Bu iki parça birbirine karışmamalıdır. Model eğitim verisiyle eğitilir, başarı ise test verisiyle ölçülür.

> **Hocanın vurgusu:** Eğitimde kullandığınız veriyi asla test için kullanamazsınız.

Hoca bunun söylenince "bundan daha doğal ne olabilir" gibi geldiğini, ama çoğu zaman yanlışlıkla eğitimde kullanılan verinin başarı ölçmek için de kullanıldığını gördüğünü söyledi. Bu durumda başarı çok yüksek çıkar. Çoğu zaman bunu yapan kişi farkında bile değildir.

İkinci adım: Eğitim verisi de iki parçaya ayrılır: eğitim ve doğrulama (validation). Akış şöyledir:

1. Elinizdeki veriyi eğitim ve test için birbirine değmeyen iki parçaya ayırın.
2. Eğitim verisini de eğitim ve doğrulama olarak ikiye ayırın.
3. Modeli eğitim verisiyle oluşturun. Örneğin karar ağacını eğitim verisiyle kurun.
4. İnce ayarları (tuning) doğrulama verisiyle yapın. Ağacı budayacak mısınız, k-NN'de K 3 mü olacak 5 mi, bunlara doğrulama verisiyle karar verin.
5. "Tamam, artık oldu" dediğiniz noktada, test verisini hiçbir şeye dokunmadan yalnızca başarı ölçmek için kullanın ve "bu karar ağacı modelimin başarısı şudur" deyin.

Hoca bunlara "altın kurallar" dedi ve çok sık hatalı yapıldığını tekrarladı. Slayttaki ifadeyle bu kümeler birbirinden ayrık (disjoint) olmalıdır.

7.1 Aynı Veri Hiç Mi Kullanılmayacak?

Hoca burada ince bir noktayı açıkladı. Bir örnek, farklı aşamalarda eğitimde de, doğrulamada da, testte de kullanılabilir. Kural şudur: Aynı aşamada, aynı seansta, aynı veri hem eğitim hem doğrulama hem test için kullanılamaz. Bir turda doğrulama için kullanılan örnek başka bir turda eğitimde yer alabilir. Slayttaki "bu işlem defalarca tekrarlanır ve ortalama hata elde edilir" ifadesi bunu anlatır; aşağıda göreceğimiz yöntemlerin temeli budur.

Tüm akış slaytta şöyle özetlenmişti: Veri eğitim ve test olarak bölünür; eğitim verisi doğrulama verisini de içerir. Eğitim kümesiyle (training set) model oluşturulur. Doğrulama kümesiyle (validation set) parametrelerle oynanarak en iyi model elde edilir. Öğrenilmiş model, test kümesiyle (test set) ölçülür. Sınıflandırıcı oluşturulurken test verisi hiçbir şekilde kullanılmaz. Gerekirse sonuçlara bakıp modele dönülebilir, örneğin "özellikleri azaltsam mı?" diye düşünülebilir; ama test verisi ayar için karıştırılmaz.

8. Az Veri Problemi

Tam bu noktada bir öğrencinin sorusu vardı: Veri hangi oranda bölünmeli? Hoca soruyu, bölme yöntemlerini anlatırken cevaplayacağını söyledi ve önce sorunun neden önemli olduğunu açıkladı.

Genellikle en büyük problemimiz veri kümesini toplamaktır. Büyük bir veri kümesi toplamak zordur. Örneğin tıbbi bir problem üzerinde çalışıyorsanız zar zor 50 hasta bulursunuz. Bunun bir kısmını eğitimde, bir kısmını doğrulamada, bir kısmını testte kullanırsanız, zaten az olan veri daha da küçük parçalara bölünür. Oysa genelleme yapmak istiyorsunuz; veriyi o kadar bölerseniz eğitimde genellemeye gitmekte zorlanırsınız. Eğitim verisi ne kadar çok olursa model o kadar iyi olur. Bu yüzden veriyi eğitim, doğrulama ve test için en uygun şekilde bölmeye çalışmamız gerekir. Bunun için çeşitli yöntemler vardır.

9. Holdout Yöntemi

İlk yöntem holdout yöntemidir. Elinizdeki verinin üçte ikisi $(2/3)$ eğitim için, üçte biri $(1/3)$ test için ayrılır. Test kısmı ellenmez. Eğitim için ayrılan $2/3$'lük kısım da yine eğitim ve doğrulama olarak ikiye ayrılır. Dersin ikinci yarısında hoca, başka oranların da kullanıldığını ekledi: Bazı çalışmalarda %80 eğitim, %20 test gibi bölünür. Genel ilke, eğitimin ağırlıklı, testin daha az olmasıdır.

Holdout ile akış özetle şöyledir: Eğitim örnekleriyle model oluşturulur. Doğrulama örnekleriyle modelin parametreleri belirlenir (k-NN ise K kaç olacak, karar ağacı ise ağaç ne kadar budanmalı). Birden fazla modeliniz varsa (karar ağacı, k-NN, yapay sinir ağı…) hepsi için en iyi parametreler belirlenir, en iyi model seçilir. Sonra eğitime hiç karışmamış test verisiyle başarı ölçülür ve "sistemimizin başarısı budur" denir.

Holdout'un sıkıntısı, elinizdeki veri küçükse ortaya çıkar. Veri üç parçaya bölünür ve her parça daha da küçülür.

> **Ek bilgi (derste anlatılmadı):** 150 örneklik bir veri kümesinde holdout ile $2/3$ oranında bölme, 100 örneği eğitime, 50 örneği teste ayırır. Eğitim kısmı da örneğin %80-%20 ile bölünürse 80 örnek eğitimde, 20 örnek doğrulamada kalır. Az veride bu kadar parçalamanın neden sıkıntılı olduğu bu sayılardan görülebilir.

10. Rastgele Alt Örnekleme

Veri küçükse yapılabilecek ilk şey rastgele alt örneklemedir (random subsampling). Veri yine $2/3$ eğitim, $1/3$ test olarak bölünür. Fakat bu işlem farklı rastgele seçimlerle defalarca tekrar edilir. Her aşamada rastgele $2/3$'lük kısım eğitim için, rastgele $1/3$'lük kısım test için seçilir ve her aşamada model yeniden oluşturulur.

İşlem K kez yapıldıysa ve E_i, i. aşamadaki hata ise, sistemin hatası ortalama hata olarak verilir:

$$E = \frac{1}{K} \sum_{i=1}^{K} E_i$$

Hoca bu yöntemin sıkıntılarını şöyle sıraladı:

- Rastgele seçimde aynı örnek defalarca eğitim tarafına düşebilir, hiç test tarafına düşmeyebilir; ya da tersi olabilir. Yani bazı örnekler hiç eğitimde ya da hiç testte yer almayabilir.
- Özellikle veri azsa, şans eseri eğitimde hep iyi örnekler, testte hep kötü örnekler olabilir; ya da tersi. Kötü örneklerle eğitilen model genellemeye gidemez.

Hoca, veri kümesi büyükse bu yöntemin sıkıntısız kullanıldığını da belirtti. Sorunlar az veride ortaya çıkar. Bütün örneklerin hem eğitimde hem testte bulunabilmesini sağlayabilirsek daha doğru olur. Bunun için çapraz doğrulama kullanılır (Bölüm 13).

11. Eğitim Örneklerine Müdahale Etmemek

İyi ve kötü örnek konusu açılmışken hoca önemli bir uyarı yaptı:

> **Hocanın vurgusu:** Veri kümenizdeki örneklere müdahale etmeniz doğru değildir.

Eski çalışmalarda eğitim örneklerinin hep çok iyi örneklerden seçilmesine çalışılırdı ve bunun iyi olduğu düşünülürdü. Sonradan doğru olmadığı anlaşıldı. Örneğin bir yüz tanıma sisteminde eğitim örnekleri için kişinin tam karşıdan bakması, ışığın harika olması, gözlerin açık olması, ağzın belli bir şekilde durması gibi şeylere dikkat edilirdi.

Soru: Hoca sordu: Bu kadar harika örneklerle eğitirsek sıkıntı ne olur? **Cevap:** Bir öğrenci, kötü bir örnek geldiğinde modelin onu tanıyamayacağını söyledi. Hoca bunu onayladı: Gelen örnek, eğitim verisindeki karakteristikten uzaklaştıkça başarı düşer.

Bu yüzden günümüzde, karakteristiği yakalamak için eğitimde "elle cımbızla tek tek seçilmiş" örnekler yerine müdahale edilmemiş, doğal örneklerin kullanılmasına odaklanılır.

12. Tabakalı Örnekleme (Stratification)

Hoca çapraz doğrulamaya geçmeden önce, bölmelerin tabakalı (stratified) yapılması gerektiğini anlattı. Slayttaki ifade "subsets are stratified before the cross-validation is performed" idi.

Örnek olarak kredi kartı sahtekârlığı tespit sistemini düşünelim. İki sınıf var: A sınıfı normal işlemleri (transaction), B sınıfı sahtekârlık yapılan işlemleri temsil ediyor. Diyelim ki bir takım dengeleme işlemleri yaptınız ve her 1 sahtekârlık örneğine karşılık 10 normal örnek olacak şekilde verinizi hazırladınız. Bu oranla başarılı bir model kurabileceğinizi düşünüyorsunuz.

Hem holdout'ta hem çapraz doğrulamada veri döndürülerek kullanılır: Bir kısmı eğitimde, bir kısmı testte yer alır. Veriyi bölerken sınıflara hiç bakmadan, rastgele ya da sırayla seçerseniz şu olabilir: Eğitim verisinin içinde 10'a 1 oranı yerine, örneğin 20'ye 1 oranı oluşabilir. Yani 20 normal örneğe karşılık 1 sahtekârlık örneği. Bu da istediğiniz sonucu vermez.

Tabakalı örnekleme bunu önler: Eğitim verisini oluştururken de test verisini oluştururken de sınıfların oranı korunur. 10 normal seçiyorsanız 1 sahtekârlık seçersiniz. Çapraz doğrulamada parçalara bölerken, eğitim için ayrılan $K-1$ parçanın içinde de normal sınıfa ait örnek sayısı 10 ise sahtekârlık sınıfına ait örnek sayısı 1 olur.

> **Hocanın vurgusu:** Buna dikkat edin, bu çok önemli. Sınıfların bulunma oranına bakmadan gelişigüzel seçerseniz, farkına varmadan oran bozulur.

Hoca, anlatınca "tabii ki böyle yapılmalı" gibi gelen bu tür detayların gerçek uygulamada çoğu zaman fark edilmediğini söyledi. Problem yaşandıkça "ben bunu 10'a 1 diye planlamıştım, eğitim yaparken bu oran nasıl bozuldu?" dediğiniz noktada fark edersiniz; o zamana kadar başarı oynamış olur.

> **Hocanın vurgusu:** Makine öğrenmesiyle ilgili çalışmalarda bütün olay detaylarda saklıdır. Genel bilgi herkesin iyi bilmesi gereken bir şeydir, ama gerçek dünya uygulamalarında başarı detaylarda saklıdır.

Bu bölümün ardından derse 15 dakika ara verildi.

13. K Katlı Çapraz Doğrulama

Aradan sonra hoca holdout yöntemini kısaca tekrar etti (Bölüm 9) ve sıkıntılarını hatırlattı: Veriyi üç parçaya ayırmak az veride sıkıntılıdır; rastgele seçimde de eğitimdeki veriler teste, testtekiler eğitime girmeyebilir ve az örnekte iyi bir genellemeye gidilemeyebilir. Bunun için önerilen yaklaşım çapraz doğrulamadır (cross-validation).

13.1 Temel Fikir

K katlı çapraz doğrulamada (k-fold cross-validation) eldeki veri K parçaya bölünür. Her adımda bir parça test, kalan $K-1$ parça eğitim için kullanılır. İşlem K kez tekrarlanır; böylece her örnek hem eğitimde hem testte yer alır. Sonunda ortalama hata hesaplanır.

Örnek 1 (slayttaki 5 katlı örnek):

- Verilen: 100 örnek, $K = 5$.
- Her parçada $100/5 = 20$ örnek vardır.
- Birinci turda birinci parça test, kalan 4 parça (80 örnek) eğitim için kullanılır. İkinci turda ikinci parça test olur, ve böyle devam eder.
- Her turda test parçasındaki 20 örnek için doğruluk (accuracy) hesaplanır. Hocanın verdiği basit örnek: 20 örnekten 11'ini doğru bulduysanız o turun doğruluğu $11/20 = 0{,}55$, yani %55'tir.
- Beş turun ortalaması alındığında slayttaki örnekte sistemin başarısı %73 bulunmuştu.

13.2 İki Kullanım Biçimi

Hoca çapraz doğrulamanın iki şekilde kullanıldığını anlattı:

- **Ayrı test verisi olmadan:** Veri K parçaya bölünür, döndürülerek eğitim ve doğrulama yapılır, ortalama hata verilir. Ayrı bir test verisi kullanılmaz.
- **Ayrı test verisiyle:** Test verisi önce ayrılır ve hiç karıştırılmaz. Çapraz doğrulama yalnızca kalan veri üzerinde yapılır. Burada her turdaki parça doğrulama verisidir. Farklı parametreler için başarı ölçülür, en iyi model seçilir. En sonunda dışarıda bırakılan test verisiyle son başarı verilir.

13.3 Çapraz Doğrulama ile Parametre Seçimi

Örnek 2 (k-NN için K seçimi): k-NN'de K'nin 1, 3, 5 ve 7 değerlerini deneyeceğinizi düşünelim. Bu durumda K katlı çapraz doğrulamayı her aday değer için ayrı ayrı yaparsınız:

1. Test verisini ayırın ve kenara koyun.
2. Kalan veriyi örneğin 5 parçaya bölün.
3. $K = 3$ için: 4 parçayla eğitin, 1 parçayla doğrulayın, başarıyı ölçün. Bunu 5 parçanın her biri doğrulama olacak şekilde tekrarlayın ve ortalama başarıyı bulun. Böylece $K = 3$ için bir ortalama başarı elde edersiniz.
4. Aynı işlemi $K = 1$, $K = 5$, $K = 7$ için tekrarlayın.
5. Ortalama başarısı en iyi olan K değerini seçin.
6. Sonra, dışarıda bıraktığınız test verisiyle bu en iyi modelin başarısını ölçün.

Burada iki farklı "K" olduğuna dikkat edin: Biri k-NN'in komşu sayısı, diğeri çapraz doğrulamanın parça sayısı.

13.4 Kaç Parçaya Bölmeli?

Parça sayısı genelde 5 ya da 10 alınır. Elinizde yeterince örnek varsa en çok kullanılan, veriyi 10 parçaya bölen 10 katlı çapraz doğrulamadır (10-fold cross-validation).

14. Birini Dışarıda Bırak ve Bootstrap

14.1 Birini Dışarıda Bırak (Leave-One-Out)

Veri yetersizse birini dışarıda bırak çapraz doğrulaması (leave-one-out cross-validation) kullanılır. Veri N örnekten oluşuyorsa, her adımda $N-1$ örnekle eğitilir, kalan 1 örnekle doğrulanır. Böylece:

- Bütün örnekler hem eğitimde hem doğrulamada yer alır.
- Az veri, eğitim, doğrulama ve test için çok fazla parçalanmamış olur.

Bu, aslında $K = N$ alınmış K katlı çapraz doğrulamadır.

14.2 Bootstrap

Veriyi eğitim ve test için ayırmanın bir başka yolu bootstrap yöntemidir. Burada da rastgele seçim yapılır, ama şu farkla:

- Normalde N örneğiniz varsa, eğitim için yine N örnek seçersiniz.

- Seçim yerine koyarak (with replacement) yapılır: Aynı örnek birden fazla kez seçilebilir.
- Hiç seçilmemiş örnekler test için kullanılır.
- İşlem yine K kez tekrarlanır.

Hocanın örneğinde 5 örnek vardı: Eğitim için rastgele 5 örnek seçilir, aynı örnek birden fazla gelebilir, seçilmeyenler teste gider.

> **Ek bilgi (derste anlatılmadı):** N örnekten yerine koyarak N seçim yapıldığında, belli bir örneğin hiç seçilmeme olasılığı $(1-1/N)^N$'dir. Bu değer $N = 5$ için yaklaşık 0,33, $N = 100$ için 0,366'dır ve N büyüdükçe $e^{-1} \approx 0{,}368$'e yaklaşır. Yani eğitime ortalama verinin yaklaşık %63,2'si girer, %36,8'i teste kalır. Bu yüzden yöntem literatürde "0.632 bootstrap" adıyla da anılır.

14.3 Hangisi Ne Zaman?

Hoca uygulamadaki pratiği şöyle özetledi: İyi, yeterli büyüklükte bir veri kümeniz varsa genelde %60-%70 eğitim, %30 test olacak şekilde ikiye bölersiniz; eğitim kısmını da eğitim ve doğrulama olarak ayırırsınız. Çok fazla örneğiniz yoksa çapraz doğrulama yaparsınız. En çok kullanılan yaklaşımlar bunlardır. Bölmelerde tabakalı örneklemeye, yani farklı sınıfların oranlarını korumaya mutlaka dikkat edilmelidir.

15. Son Model: Bütün Veriyle Yeniden Eğitmek

Hoca, bütün model oluşturma sistemlerinde genelde yapılan bir şeyi de anlattı. Modeli eğittiniz, doğruladınız, test ettiniz ve kararınızı verdiniz: Örneğin en iyi model karar ağacıdır ve en iyi budanmış ağaç da şudur. Bu kararı verdikten sonra genelde eldeki bütün veri birleştirilir ve sistem bu verinin tamamıyla yeniden eğitilir.

Sebep yine aynı hikâyedir: Örnek azdır. Siz %60'ıyla eğitip %40'ıyla test ettiniz. Ama bütün parametrelere karar verdikten sonra (örneğin k-NN'de K = 3), artık hiçbir veriyi boşa harcamamak için modeli bütün verilerle oluşturursunuz.

Dersin ilerleyen kısmında bir öğrenci bu konuyu tekrar sordu.

Soru: Modeli eğitim ve test olarak böldük, test ettik, parametreleri de optimize ettik, her şeyden memnunuz. Gerçek hayata çıkarken eğittiğimiz kısımla oluşan modeli mi kullanıyoruz, yoksa aynı parametrelerle verinin %100'ünü kullanıp yeniden mi eğitiyoruz?

Cevap: Hoca bunun az önce söylediği konu olduğunu belirtip tekrar etti. Bütün parametreler belirlenir. Bunlar yalnızca K gibi değerler değildir; örneğin ileriki haftalarda görülecek yapay sinir ağlarında kaç katman olacağı gibi başka parametreler de vardır. En iyi model ve parametreler belirlenip sabitlendikten sonra eğitim, doğrulama ve test verilerinin hepsi birleştirilir, sistem bununla eğitilir ve gerçek dünyaya öyle verilir. Çok örneğiniz varsa buna gerek olmayabilir. Örneğin bir yıllık veriyle kredi kartı sahtekârlığı üzerinde çalışıyorsanız elinizde zaten çok büyük veri vardır. Ama çalışılan problemlerin çoğunda veri toplamak büyük bir sıkıntıdır ve veri azdır. O durumda gerçek dünyaya çıkacak modeli bütün veriyle eğitmek doğru bir yaklaşımdır.

16. Dengesiz Veri

Hoca, ayrı bir derste ayrıntılı anlatacağını söyleyerek dengesiz veriden (imbalanced data) de kısaca bahsetti.

Gerçek dünya problemlerinde veri sınıflar arasında dengeli olmayabilir. Bazı problemlerde veri doğal olarak dengelidir, çünkü toplama kontrollüdür: 10 kişiyi tanıyacak bir yüz tanıma sisteminde her kişi için eşit sayıda yüz örneği toplanabilir; araba modeli tanımada da benzer şekildedir. Ama bazı problemlerde bir sınıfın örneği doğal olarak azdır:

- Hastalık teşhisinde normal veri bulmak, hastalık verisi bulmaktan her zaman kolaydır.
- Kredi kartı sahtekârlığında normal işlemler çok, sahte işlemler azdır.
- Ağ saldırısı tespitinde günlerce ağı dinlersiniz, hep normal veri gelir; saldırı verisi bulmak daha zordur.

Hocaya göre iki temel yaklaşım vardır.

16.1 Çok Olan Sınıfı Azaltmak

Çok örneği olan sınıfın örnek sayısı, az olan sınıftakine kadar azaltılabilir. Fakat bu iyi bir şey değildir, çünkü az olan sınıfta zaten az örnek vardır ve toplam veri iyice küçülür.

Azaltmayı eşit değil, oransal yapabilirsiniz. Bir saldırı örneğine karşılık kaç normal örnek olursa başarılı bir sınıflandırma yapıldığını testlerle ölçersiniz. Örneğin 1'e 5 oranında başarılı bir model oluyorsa oranı 1'e 5 tutarsınız; 1'e 10'da oluyorsa 1'e 10 tutarsınız. Yani "bir sınıftan ne kadar varsa o kadar kalsın" diye serbest bırakmazsınız. Çünkü örneğin saldırı sınıfından 1, normal sınıftan 100 örnek varsa öbür sınıf çok baskın olur ve ciddi bir genelleme sıkıntısı yaşarsınız.

16.2 Veri Artırma

İkinci yaklaşım veri artırmadır (data augmentation). Hoca bunun, derin öğrenme modellerinin başarısının yüksek olmasıyla birlikte şu anda çok fazla kullanıldığını söyledi ve son derslerde derin öğrenmeye bir özet olarak değineceğini belirtti.

- **Üretici çekişmeli ağlar (Generative Adversarial Networks, GAN):** Çok iyi sahte veri üretirler. Az olan sınıf için sahte örnekler üreterek örnek sayısı artırılabilir.
- **Aynı örneği değiştirerek kullanmak:** Örneğin bir görüntü verisinde görüntüyü biraz döndürerek, bir kısmını keserek yeni örnekler elde edilebilir.

16.3 Model Değerlendirme Özeti

Hoca bu kısmı slayttaki özetle bağladı:

1. Çok örneğiniz varsa veriyi eğitim, doğrulama ve test için ayırın.
2. Dengesiz veride dengeleme yapın: oransal azaltma ya da veri artırma.
3. Az veriniz varsa holdout yerine çapraz doğrulama kullanın.
4. Parametreleri belirlemek için test verisini hiçbir zaman işe karıştırmayın. Testte yalnızca başarı ölçün.
5. En önemlisi aşırı öğrenmeden kaçının. Eğitim başarısı çok yüksek, test başarısı çok düşükse aşırı öğrenme olmuştur ve modelinizi tekrar gözden geçirmeniz gerekir.

17. Karışıklık Matrisi

Buraya kadar hangi modelin daha iyi olduğuna karar verme sürecini gördük: Örneğin k-NN %90, karar ağacı %91,5, yapay sinir ağı %92 başarı verdiyse yapay sinir ağını seçersiniz. Model değerlendirmede (model evaluation) yaptığımız şey budur: Doğrulama testleriyle en uygun modeli ve onun en uygun parametrelerini belirlemek. Şimdi asıl soru: Bu karar için başarıyı nasıl ölçmeliyiz?

Hoca önce iki sınıflı bir problemle başladı ve mutlaka bir karışıklık matrisi (confusion matrix) oluşturulması gerektiğini söyledi. Diyelim ki iki sınıfımız var: evet ve hayır (yes / no). Bu bir ikili sınıflandırmadır (binary classification). Normal harcama ve sahte harcama, normal ağ hareketi ve saldırı, hasta ve sağlam gibi pek çok problem zaten iki sınıflıdır.

	Model: pozitif	Model: negatif
Gerçek: pozitif	Doğru pozitif (TP)	Yanlış negatif (FN)
Gerçek: negatif	Yanlış pozitif (FP)	Doğru negatif (TN)

- **Doğru pozitif (true positive, TP):** Gerçek sınıf pozitif, model de pozitif diyor.
- **Yanlış negatif (false negative, FN):** Gerçek sınıf pozitif, ama model negatif diyor. Pozitif olması gerekirken negatif denmiştir.
- **Yanlış pozitif (false positive, FP):** Gerçek sınıf negatif, ama model pozitif diyor.
- **Doğru negatif (true negative, TN):** Gerçek sınıf negatif, model de negatif diyor.

Burada istenen, köşegenin (diagonal) yüksek olmasıdır, yani doğru sınıflandırma sonuçlarının yüksek olması. Ama hoca tek başına bunun önemli olmadığını vurguladı: Yanlışların az olması da önemlidir.

17.1 COVID Testi Tartışması

Hoca bunu o günlerde gündemde olan COVID testleriyle açıkladı. Herkes testlerin "%40 doğru çıktığını" söylüyordu. Yani konuşulan, pozitif olanların %40'ına pozitif dendiğiydi; bu doğru pozitiflerle ilgili bir oran. Ama doğru negatifler hiç konuşulmuyordu: Negatif olanların yüzde kaçına negatif deniyor? Hoca bunu hâlâ bilmediğimizi söyledi. Yanlışların ne kadar olduğu da konuşulmuyordu.

Probleminize göre yalnızca doğru pozitiflerin değil, doğru negatiflerin ve yanlışların ne kadar olduğu da önemlidir. Sizin için neyin önemli olduğuna sizin karar vermeniz gerekir.

18. Çok Sınıflı Problemlerde Karışıklık Matrisini Okumak

İki sınıflı bir problemde karışıklık matrisinin yorumu kolaydır: Model doğru demediyse yanlış demiştir, her şey ortadadır. Çok sınıflı bir problemde ise karışıklık matrisi size **hangi sınıfın hangi sınıfla karıştığı** hakkında bilgi verir ve bu önemli bir bilgidir.

Hoca slaytta bir hareket tanıma (activity recognition) çalışmasının karışıklık matrisini gösterdi. Satırlarda gerçek sınıflar, sütunlarda tahmin edilen (predicted) sınıflar vardı. Çoğu sınıfta %100 doğru sınıflandırma varken, model koşmayı (run) atlamayla (skip), "wave 1" hareketini de "wave 2" ile karıştırıyordu.

Hocaya göre model kurarken iş "başarıyı ölçtüm" diye bitmez. Geri dönüp şuna bakmak gerekir: Model neden koşmayla atlamayı ya da wave 1 ile wave 2'yi karıştırıyor? Karışıklık matrisi bunu görmek için kullanılır; bakarsınız ve o problemi çözmeye odaklanırsınız. Olası sonuçlar şunlardır:

- **İki sınıf aslında birleşmeli:** Wave 1 ile wave 2 birbirine çok benzeyen hareketler olabilir. Hatayı düşürmek için değil, bu kadar ayrıntıda ayırmak doğru olmadığı için iki sınıfı birleştirmeniz gerekebilir.
- **Özellik sorunu:** Belki yanlış bir özellik kullandınız ya da kullanmanız gereken bir özelliği kullanmadınız.
- **Çözümü yok:** Ne yaparsanız yapın bu iki sınıf karışacaktır. Ama bunu görmüş ve bu kararı bilerek vermiş olursunuz.

  Hocanın vurgusu: Çok sınıflı problemlerde mutlaka karışıklık matrisine bakın, hangi sınıfın hangisiyle karıştığını görün ve neden karıştığını anlamaya çalışın.

Bunun için yanlış sınıflanan örneklere tek tek bakılmalıdır. Örneğin 11 örnek karışmışsa o 11 örneği inceleyin. Oradan şunları yakalayabilirsiniz:

- O örneklerde ortak bir problem olabilir. Bir özelliğin az ya da çok kullanıldığını anlayabilirsiniz.
- O örnekler çok kötü örnekler olabilir; ne yapsanız doğru sınıflandırılamayacaktır. "İnsan gözü bile bunu doğru sınıflandıramaz" diyebilirsiniz. Probleminize göre bu örnekleri kullanmamaya ya da durumu kabul etmeye karar verirsiniz.

18.1 Hazır Kütüphaneler ve Lisansüstü Çalışmalar

Hoca en çok gördüğü hatanın yeni sistemlerde yapıldığını söyledi. Artık çok sayıda hazır kütüphane var. Derin öğrenmeyle sabah oturup akşama yüz tanıyan bir sistem yazabilirsiniz; olay, aracı (tool) iyi ve hızlı kullanma becerinize bağlıdır. Ama sıkıntı şudur: Başarı ölçülüyor, "%90 başarı" deniyor, fakat başarısızlığın sebebinin ne olduğuna bakan yok.

> **Hocanın vurgusu:** Lisansüstü öğrencileri, özellikle doktora öğrencileri, sıfırdan kimsenin yapmadığı bir uygulama üzerinde çalışmıyorsa, var olan çalışmaların "gelecek çalışmalar" (future work) kısmında bahsedilen problemlere bakmalı ve o problemleri çözmeye yönelmelidir.

Bu problemler genellikle karışıklık matrislerinden çok rahat anlaşılır. Bir modelin neyi yapamadığını, nerede yetersiz kaldığını anlamak için bakılacak yerlerden biri karışıklık matrisidir.

19. Başarı Ölçüleri: Accuracy, Precision, Recall, Specificity, F1

19.1 Doğruluk (Accuracy)

Örnekleriniz çoksa ve sınıflara dengeli dağılmışsa, kullanacağınız en temel başarı ölçüsü doğruluktur (accuracy). Doğruluk, bütün örneklerin kaçını doğru sınıflandırdığınızı verir; yani doğru pozitiflerle doğru negatiflerin bütün örneklere oranıdır:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Buradan modelinizin ne kadar iyi olduğunu gayet iyi anlayabilirsiniz. Fakat örnekler sınıflara dengeli dağılmadıysa doğruluk iyi bir ölçüm şekli değildir.

Örnek 3 (dengesiz veride doğruluk tuzağı):

- Verilen: İki sınıflı bir problem. A sınıfında 9.990, B sınıfında 10 örnek var; toplam 10.000 örnek.
- Model: 10.000 örneğin tamamına "A sınıfı" diyor. Yani aslında hiçbir şey öğrenmemiş, kötü bir sınıflandırıcı.
- İşlem: Doğru sınıflanan örnekler A sınıfının 9.990 örneğidir. Accuracy $= 9990/10000 = 0{,}999$.
- Sonuç: Doğruluk %99,9 çıkar. B sınıfında zaten yalnızca 10 örnek olduğu için, her şeye A demek doğruluğu neredeyse hiç etkilemez.

  Hocanın vurgusu: Bu çok sık yapılan bir hatadır. Denge yoksa ve doğrulukla yüksek başarı elde ediyorsanız, belki de aslında sınıflandırma yapamayan bir sisteminiz vardır ve bunu anlayamazsınız. O zaman başka ölçülere bakmak gerekir.

19.2 Kesinlik (Precision) ve Duyarlılık (Recall)

Bu iki ölçünün anlatımında derste bir karışıklık yaşandı ve hoca bunu aradan sonra düzeltti. Not, düzeltilmiş hâli verir.

Hoca önce kesinlik için "bütün pozitifler içinde doğru bildiklerinizin oranı" dedi ve şu örneği verdi: COVID olan 1.000 kişi var, sistem bunların 500'ünü doğru bildi; bu durumda kesinlik %50'dir. Sonra duyarlılık formülüne geçince slaytta yanlış yazdığını düşünüp düzelteceğini söyledi. Bunun üzerine bir öğrenci araya girdi.

Soru: Hocam, slayttaki doğru değil mi? Pozitiflerin tamamı doğru pozitif artı yanlış negatif olmuyor mu? **Cevap:** Hoca öğrencinin haklı olduğunu söyledi: Gerçek pozitiflerin tamamı $TP + FN$'dir. Slayttaki paydayı kısaltılmış yazdığı için bir an karıştırdığını belirtti. Aradan sonra slayta açık hâlini ekledi ve iki ölçüyü yeniden anlattı.

Düzeltilmiş tanımlar şöyledir:

- **Kesinlik (precision):** Modelin pozitif dediklerinin ne kadarı gerçekten pozitif? Payda, modelin pozitif dedikleridir: $TP + FP$.

$$\text{Precision} = \frac{TP}{TP + FP}$$

- **Duyarlılık (recall), diğer adıyla sensitivity ya da doğru pozitif oranı (true positive rate, TPR):** Gerçek pozitiflerin ne kadarı doğru bilindi? Payda, gerçek pozitiflerin tamamıdır: $TP + FN$.

$$\text{Recall} = \text{Sensitivity} = \text{TPR} = \frac{TP}{TP + FN}$$

- **Özgüllük (specificity), diğer adıyla doğru negatif oranı (true negative rate, TNR):** Gerçek negatiflerin ne kadarı doğru bilindi?

$$\text{Specificity} = \text{TNR} = \frac{TN}{TN + FP}$$

Örnek 4 (aradan sonraki düzeltilmiş COVID örneği):

- Verilen: Gerçekte 1.000 COVID hastası var. Sistem bu hastaların 500'ünü doğru biliyor. Hocanın ikinci anlatımına göre sistem 500 kişiye "pozitif" diyor ve bunların 200'ü gerçekten pozitif.
- Duyarlılık: Gerçek 1.000 hastanın 500'ü doğru bilindiği için Recall $= 500/1000 = 0{,}5$.
- Kesinlik: Pozitif denen 500 kişiden 200'ü gerçekten pozitif olduğu için Precision $= 200/500 = 0{,}4$.

Hoca, ilk anlatımdaki "1.000 hastanın 500'ünü doğru bildi" örneğinin aslında duyarlılığı anlattığını, kesinliğin ise "doğru dediklerinin ne kadarı doğru" sorusunun cevabı olduğunu bu ikinci anlatımda netleştirdi.

19.3 Hangisi Daha Önemli?

Hocaya göre bazı çalışmalarda kesinlik, bazılarında duyarlılık daha önemlidir; bu sizin çalışmadan ne beklediğinize bağlıdır. Bazen gerçek pozitiflerin ne kadarını bildiğinizi, bazen de pozitif dediklerinizin ne kadarının doğru olduğunu bilmek istersiniz.

Soru: Hoca sordu: COVID meselesinde, yani hasta olanların oranını bilmek istediğimizde, sizce hangisi daha iyi? **Cevap:** Bir öğrenci "recall mı hocam?" diye sordu. Hoca bunun cevabını iki ölçünün de kullanım şeklini anlatarak verdi:

- **Kesinlik açısından:** 100 kişiye pozitif dediniz ve bunların 50'si doğruysa, 50 kişiye yanlışlıkla pozitif demişsiniz. Slayttaki ifadeyle kesinlik, "yanlış pozitifin maliyeti yüksek olduğunda" (the cost of false positive is high) faydalıdır. Hocanın örneği: Hasta olmayan 50 kişiye "kansersin" deyip kanser tedavisi uygulamak büyük bir sıkıntıdır.
- **Duyarlılık açısından:** Gerçekte 100 kişi hasta ve siz bunların yalnızca 20'sini bilebiliyorsunuz. 80 hastayı kaçırmışsınız. COVID meselesinde bu 80 kişi hastadır; hastalık ilerleyip akciğere indiğinde fark edilecektir. Burada sıkıntı, kaçırılanlardan kaynaklanır.

Bu yüzden pek çok çalışmada hem kesinliğin hem duyarlılığın bilinmesi uygundur.

19.4 F1 Skoru

Kesinlik ve duyarlılığı birlikte kullanan ölçü F ölçüsüdür (F-measure). İkisinin harmonik ortalamasıdır ve F1 skoru (F1 score) olarak da adlandırılır:

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

Çalışmalarda yalnızca kesinlik ya da yalnızca duyarlılık değil, F1 skoru da birlikte verilir.

> **Ek bilgi (derste anlatılmadı):** Hocanın iki örneğindeki değerleri birleştirip kesinliği 0,5, duyarlılığı 0,2 olan bir model düşünelim. Aritmetik ortalama 0,35 olurdu. F1 ise $2 \cdot (0{,}5 \cdot 0{,}2)/(0{,}5 + 0{,}2) = 0{,}2/0{,}7 \approx 0{,}286$'dır. Harmonik ortalama küçük olan değere daha yakın durur; bu yüzden iki ölçüden biri kötüyse F1 de düşük çıkar.

19.5 Accuracy mi, F1 mi?

Hoca slayttaki kuralı şöyle açıkladı:

- Doğru pozitifleri ve doğru negatifleri yakalamak daha önemliyse ve sınıflarda yaklaşık eşit sayıda örnek varsa doğruluk (accuracy) kullanmak doğrudur.
- Kaçırdıklarınızı bilmek sizin için önemliyse F1 skoru kullanılır. Yani hasta olmasına rağmen sağlam denenleri ya da hasta olmamasına rağmen hasta denenleri bilmek önemliyse, kısacası yanlış sınıflandırılanların önemi büyükse F1 skoru kullanılır.

20. ROC Eğrisi ve AUC

20.1 Tarihçe ve Tanım

Hoca son olarak sonuçları görselleştirmenin bir yolunu anlattı: ROC eğrisi (Receiver Operating Characteristic curve). ROC eğrisi ilk olarak İkinci Dünya Savaşı'nda radar operatörleri tarafından, radarın tespit ettiği şeyin bir nesne mi yoksa parazit mi olduğunu değerlendirmek için kullanıldı. Daha sonra tıbbi uygulamalarda, radyolojide ve görüntülemede, sonra da makine öğrenmesi uygulamalarında kullanılmaya başlandı.

ROC eğrisi, yanlış pozitif oranına (false positive rate, FPR) karşılık doğru pozitif oranının (TPR) görselleştirilmesidir. x ekseni yanlış pozitif oranı, y ekseni doğru pozitif oranıdır. Eğri, farklı eşik (threshold) değerleri için bu iki oranın durumunu gösterir.

> **Ek bilgi (derste anlatılmadı):** Yanlış pozitif oranı FPR $= FP/(FP + TN)$ olarak tanımlanır ve $1 -$ Specificity'ye eşittir. Bu yüzden ROC eğrisinin x ekseni bazen "1 − özgüllük" olarak da etiketlenir.

20.2 Eşik Değeri

Hoca bir hastalık teşhisi örneği çizdi. İki dağılım vardı: Mavi olanlar hasta kişiler, kırmızı olanlar sağlam kişiler. Bir eksen özgüllüğü, öbürü duyarlılığı temsil ediyordu.

İdeal durumda hastalar ile sağlamlar, yani doğru pozitiflerle doğru negatifler hiç birbirine karışmaz. Gerçek dünyada böyle bir durum yoktur; iki dağılım üst üste biner. Bu yüzden karar verirken genelde bir eşik değeri belirlemek zorunda kalırız:

- Eşik değerinden büyük olan herkese "hasta" (pozitif) deriz.
- Eşik değerinin altındaki herkese "sağlam" (negatif) deriz.

Eşik değeri bir yere konduğunda dört bölge oluşur: doğru pozitifler, yanlış pozitifler, doğru negatifler ve yanlış negatifler. Eşiği kaydırdıkça bu bölgelerin büyüklüğü değişir. Peki eşik nerede olmalı? ROC eğrisi bu karar için kullanılır: Farklı eşik değerleri için yanlış pozitif oranına karşılık doğru pozitif oranını görürsünüz. Örneğin bir eşikte yanlış pozitif oranı 0,2 iken doğru pozitif oranı 0,6 olabilir. İdeal nokta, özgüllüğün ve duyarlılığın birlikte yüksek olduğu, yani eğrinin sol üst köşesine yakın olan noktadır.

20.3 Eğri Altındaki Alan (AUC)

ROC eğrisinin altındaki alana eğri altındaki alan (Area Under the Curve, AUC) denir. AUC 0 ile 1 arasında bir değer alır.

- **AUC = 1:** Bütün kararlar doğrudur. Doğru pozitifler ile doğru negatifler tamamen ayrıdır; eşik tam aradan geçirilir ve kimse kimseye karışmaz. Bu ideal durumdur, gerçekte olabilecek bir şey değildir.
- **AUC = 0,7:** Hocanın örneğinde eşik bir yere konmuş ve değer %70 çıkmıştı. Bir takım negatifler pozitif tarafına, bir takım pozitifler de negatif tarafına geçmiştir. Hoca bunu "%70 oranında pozitiflerle negatiflerin ayrılabildiği" şeklinde yorumladı.
- **AUC = 0,5:** Pozitiflerle negatifler tamamen karışmıştır; model doğruları yanlışlardan hiçbir şekilde ayıramıyordur. Bu, yazı-tura atmakla, yani rastgele seçimle aynıdır. Eğri köşegen üzerindedir.
- **AUC = 0:** Bütün kararlar yanlıştır: Model bütün pozitiflere negatif, bütün negatiflere pozitif diyordur. Hoca böyle bir modeliniz varsa "bırakın gidin" dedi ve zaten böyle bir modelin büyük ihtimalle olmayacağını ekledi.

Yani ideal olan, sol üst köşeye doğru kabaran bir ROC eğrisi elde etmektir. Köşegene yaklaşan bir eğri kötü bir modeli gösterir.

20.4 ROC Eğrisi Ne İçin Kullanılır?

Hoca iki kullanım verdi:

- **Tek bir model üzerinde çalışıyorsanız:** En iyi eşik değerinin ne olduğunu belirlemek için ROC eğrisi çizilir.

- **Birden fazla modeliniz varsa:** Her model için, yanlış pozitif oranına ve doğru pozitif oranına göre ROC uzayında bir nokta oluşturulur. Örneğin bir nokta k-NN'i, bir nokta karar ağacını, bir nokta yapay sinir ağını gösterir. Bu noktaların yerine bakarak hangi modelin daha iyi olduğunu görebilirsiniz. Hocanın yorumu: Yanlış pozitif oranı %20 iken doğru pozitif oranı %80 olan bir model fena değildir. Doğru pozitif oranı artarken yanlış pozitif oranı da artıyorsa ve siz bunu tercih etmiyorsanız, ideal olan sol üst köşeye yakın noktalardır.

Hoca bu kısım için hazırladığını düşündüğü bir slaytı koymadığını fark etti ve farklı modellerin ROC uzayındaki karşılaştırmasını sözlü olarak anlattı.

20.5 Hata Ölçümü Üzerine Son Bir Not

Hoca dersin sonunda, burada anlatılan kesinlik ve duyarlılık gibi ölçülerin genel başarı ölçüleri olduğunu, ama hatanın nasıl hesaplanacağının yöntemden yönteme değiştiğini söyledi. Farklı yöntemlerin hata hesaplarından, ilgili modelleri anlatırken bahsedeceğini belirtti.

Dersin Toparlaması

Bu derste, bir modelin "iyi" olup olmadığına nasıl karar verileceğini gördük. Amaç genellemedir: Model eğitim verisini ezberlememeli, hiç görmediği örneklerde de doğru karar vermelidir. Eğitim başarısının test başarısından çok yüksek olması aşırı öğrenmenin, öğrenmenin erken kesilmesi ise yetersiz öğrenmenin işaretidir. Benzer başarıdaki modellerden basit olanı seçilir (Occam'ın usturası). Tek bir model ve tek bir parametreyle yetinilmez; birden fazla model ve parametre denenir.

Bunun için veri eğitim, doğrulama ve test olarak ayrık parçalara bölünür. Eğitim verisiyle model kurulur, doğrulama verisiyle ince ayar yapılır, test verisi yalnızca son başarıyı ölçmek için kullanılır. Az veride holdout yerine K katlı çapraz doğrulama (genelde 5 ya da 10 kat), çok az veride birini dışarıda bırak, alternatif olarak bootstrap kullanılır. Bütün bölmelerde sınıf oranları korunur (stratification). Model ve parametreler belirlendikten sonra, gerçek kullanıma çıkarken model genelde bütün veriyle yeniden eğitilir. Dengesiz veride çok olan sınıf oransal olarak azaltılır ya da veri artırma yapılır.

Başarı ölçümünde karışıklık matrisi temeldir; çok sınıflı problemlerde hangi sınıfın hangisiyle karıştığını gösterir ve yanlış sınıflanan örneklerin incelenmesine yol açar. Doğruluk dengeli veride iyi bir ölçüdür, dengesiz veride yanıltıcıdır. Kesinlik yanlış pozitiflerin, duyarlılık kaçırılan pozitiflerin maliyetini öne çıkarır; F1 skoru ikisini harmonik ortalamayla birleştirir. ROC eğrisi farklı eşiklerde doğru pozitif oranını yanlış pozitif oranına karşı gösterir; eşik seçimi ve model karşılaştırması için kullanılır, AUC ise eğrinin altındaki alanla modeli tek sayıda özetler. Hoca, farklı yöntemlerin hata hesaplarını ileriki derslerde ilgili modellerle birlikte anlatacağını söyledi.

Ders Sonrası Notlar

Ders sonrası not: Hoca kesinliği ilk anlatımında "bütün pozitif olanlar içinde doğru bildiklerinizin oranı" diye tanımladı ve "1.000 COVID hastasının 500'ünü doğru bildiyse kesinlik %50" örneğini verdi. Bu tanım ve örnek aslında duyarlılığa (recall) aittir. Doğrusu: Kesinlik $TP/(TP+FP)$, yani modelin pozitif dediklerinin ne kadarının doğru olduğudur; duyarlılık $TP/(TP+FN)$, yani gerçek pozitiflerin ne kadarının bilindiğidir. Hoca aradan sonra iki ölçüyü bu şekilde yeniden anlattı.

Ders sonrası not: Hoca özgüllüğü (specificity) anlatırken bir yerde "hasta olanların ne kadarını doğru bildi" dedi. Doğrusu: Özgüllük, gerçek negatiflerin (bu örnekte sağlam kişilerin) ne kadarına doğru olarak negatif dendiğidir: $TN/(TN+FP)$. Hoca aynı cümlenin devamında da "gerçek negatiflerin ne kadarını doğru bildi" diyerek doğru tanımı verdi.

Ders sonrası not: Hoca, COVID testlerinde konuşulan %40'lık doğru pozitif oranının "yazı-tura atmaktan (%50) daha kötü" olduğunu söyledi. Bu karşılaştırma tek başına doğru değildir. Rastgele bir tahmin, doğru pozitif oranı ile yanlış pozitif oranını

birlikte 0,5 yapar; yani ROC eğrisinde köşegen üzerinde durur. Bir testin rastgeleden iyi ya da kötü olduğunu anlamak için duyarlılığın yanında yanlış pozitif oranına (ya da özgüllüğe) da bakmak gerekir. Örneğin duyarlılığı 0,4 olan bir test, yanlış pozitif oranı 0'a yakınsa pozitif sonucu çok güvenilir olabilir; sorunu, hastaların çoğunu kaçırmasıdır. Hocanın aynı derste vurguladığı "doğru negatifler de konuşulmalı" uyarısı tam da bu yüzden önemlidir.

Ders sonrası not: Hoca AUC değeri 0,7 olan bir durumu "%70 oranında doğru sınıflandırma" olarak yorumladı. AUC bir doğruluk (accuracy) değeri değildir. AUC, rastgele seçilen bir pozitif örneğin modelden rastgele seçilen bir negatif örnekten daha yüksek skor alma olasılığıdır ve tek bir eşiğe bağlı değildir. Doğruluk ise seçilen eşiğe göre değişir.

Ders sonrası not: Hoca AUC = 0 durumu için "o modeli bırakın gidin" dedi. Böyle bir model aslında tam tersini sistematik olarak bilmektedir; çıktısı tersine çevrilirse (pozitif yerine negatif, negatif yerine pozitif denirse) AUC 1 olur. Genel olarak AUC'si 0,5'in altında kalan bir sınıflandırıcıda etiketlerin ya da skorun yönünün ters kurulmuş olabileceğine bakmak gerekir.

Ders sonrası not: Altyazıda "Occam-Schreizer" geçen ilke Occam'ın usturası (Occam's razor), "Generative Advisory Network" diye geçen yöntem Generative Adversarial Network (GAN), "ekürisi" diye geçen ölçü accuracy, "Limanat cross-validation" diye geçen yöntem leave-one-out cross-validation, "Pol Dağıt" diye geçen yöntem holdout, "Rikol" diye geçen ölçü recall, "ROCC" diye geçen kısaltma ROC'dir. "FutureBörg" ise makalelerin future work (gelecek çalışmalar) bölümüdür.

Not: İsterseniz bu kayıtları NotebookLM gibi bir araca atıp daha istifadeli hale getirebilirsiniz.